

ENSE 281 GROUP A PROJECT REPORT

BOOKEXCHANGE

Thomas Persson - 200525550

Chop Peter Kur - 200497265

Shaikh Chand Muhammad Shami - 200491628

David James-Nwokocha - 200518242

ENSE 281 - Software Engineering Management

University of Regina

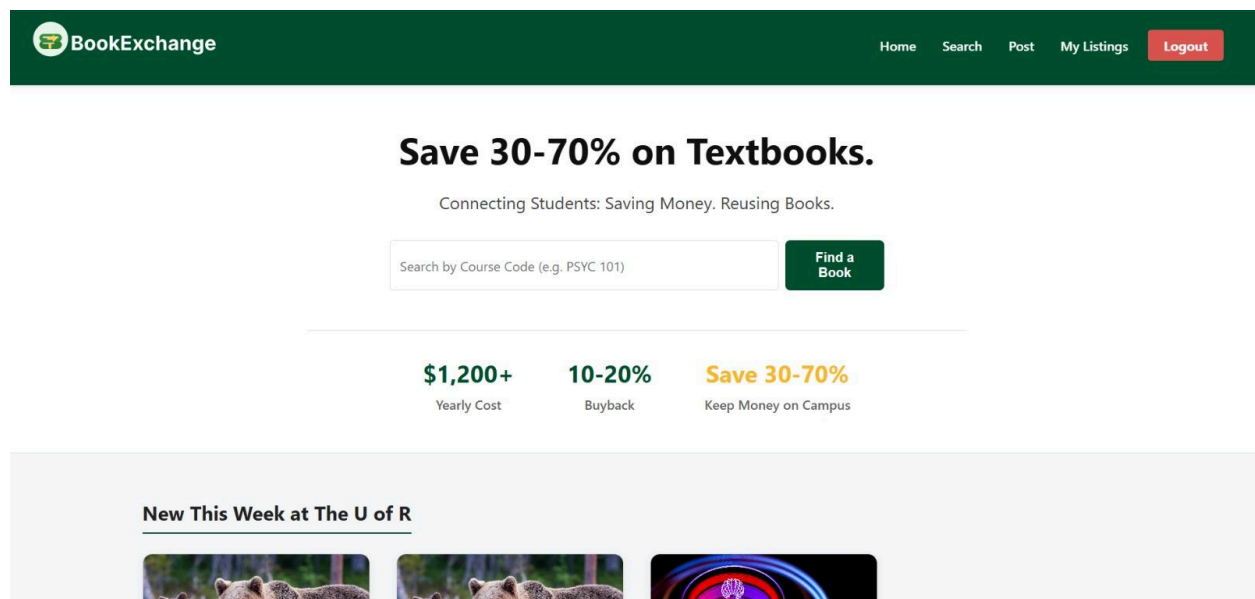
Instructor: Dr. Tim Maciag

April 2025

Introduction

BookExchange is a peer-to-peer textbook marketplace web application designed specifically for University of Regina students. The platform addresses a common pain point in student life: the high cost of textbooks and the lack of a convenient, campus-specific channel for buying and selling them. This report details the goals, development process, design decisions, feedback cycles, and outcomes of the BookExchange project, built as part of ENSE 281 - Software Engineering Management.

University textbooks can cost between \$100 and \$300 per book. The campus bookstore typically offers only 10–20% of the original price at buyback, leaving students with little financial recourse. No dedicated campus-specific platform existed to connect student buyers and sellers directly. BookExchange was conceived as a simple, accessible solution that required no backend infrastructure, making it realistic to build within a single semester.



The application was built entirely with HTML5, CSS3, and vanilla JavaScript, using the browser's LocalStorage API for data persistence. This technology stack was deliberately chosen for its low barrier to entry and its alignment with the team's current skill set, allowing all four members to contribute meaningfully throughout the project.

Process:

Project Goals

The team established the following goals at the outset of the project:

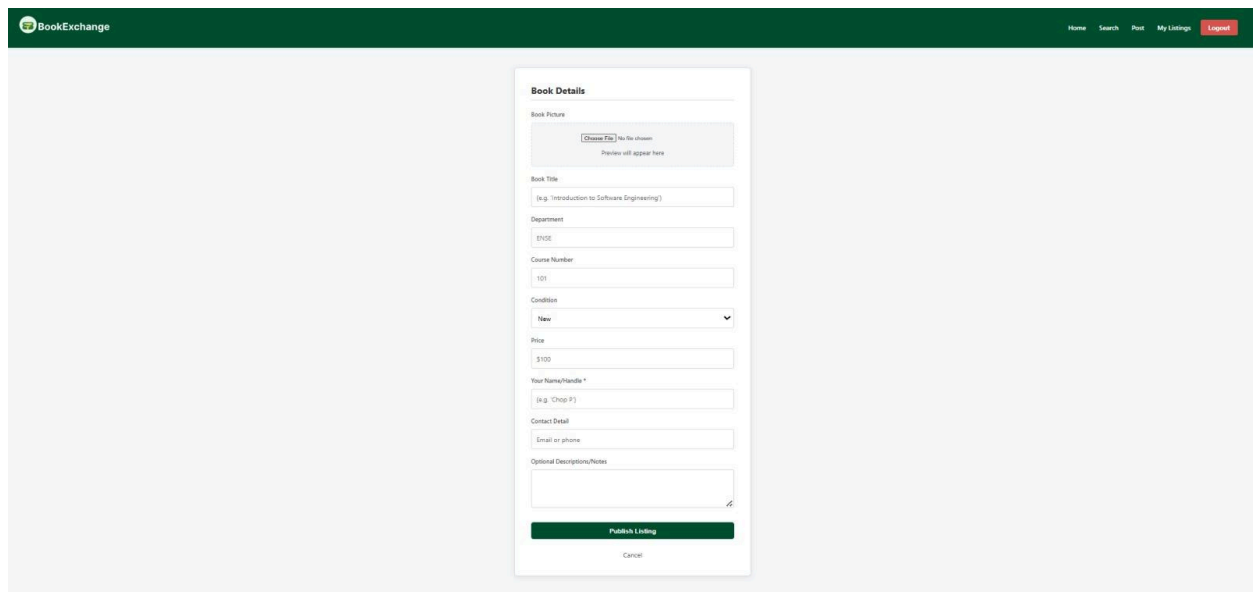
- Build a functional MVP that allows students to post, browse, search, and view textbook listings.
- Enable seller-side controls: marking a listing as sold and deleting listings.

- Keep the platform campus-specific, requiring no login or backend to reduce complexity.
- Deliver a clean, intuitive interface that could realistically be used by fellow students.

Were Goals Achieved?

Yes, all six core features were successfully implemented and are fully functional in the delivered MVP:

- Post a book listing (title, course code, condition, price, contact info)

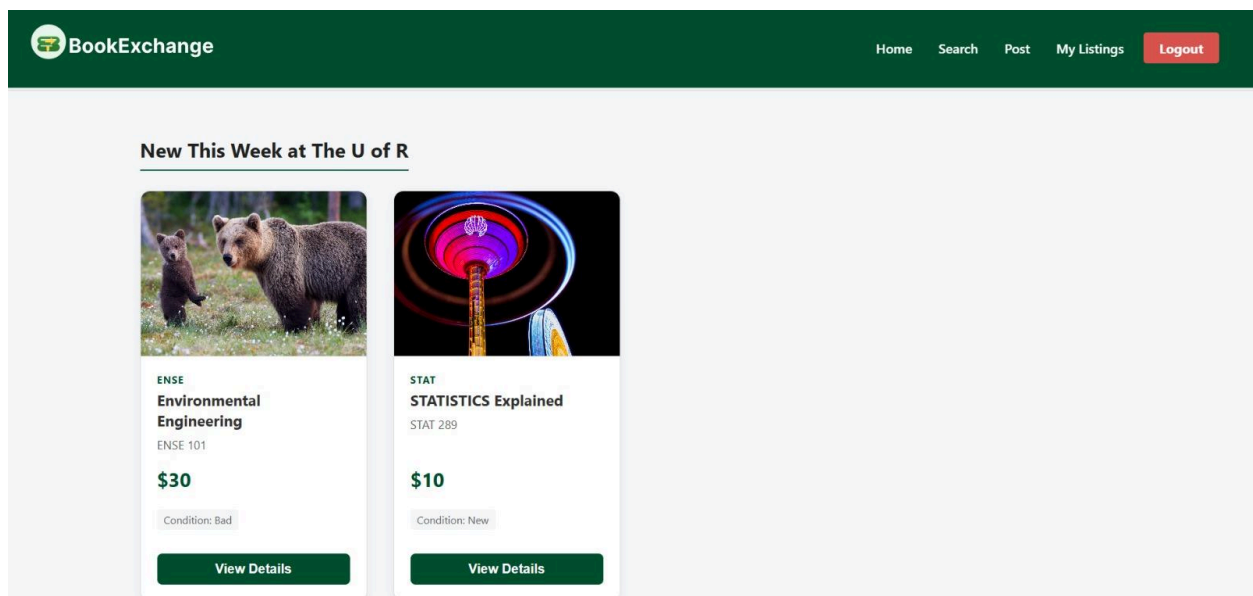


The screenshot shows the 'Book Details' form in the BookExchange MVP. The form is centered on a light gray background. At the top, there's a dark green header with the 'BookExchange' logo and navigation links: Home, Search, Post, My Listings, and Logout. The form itself has a white background and a green border. It contains the following fields:

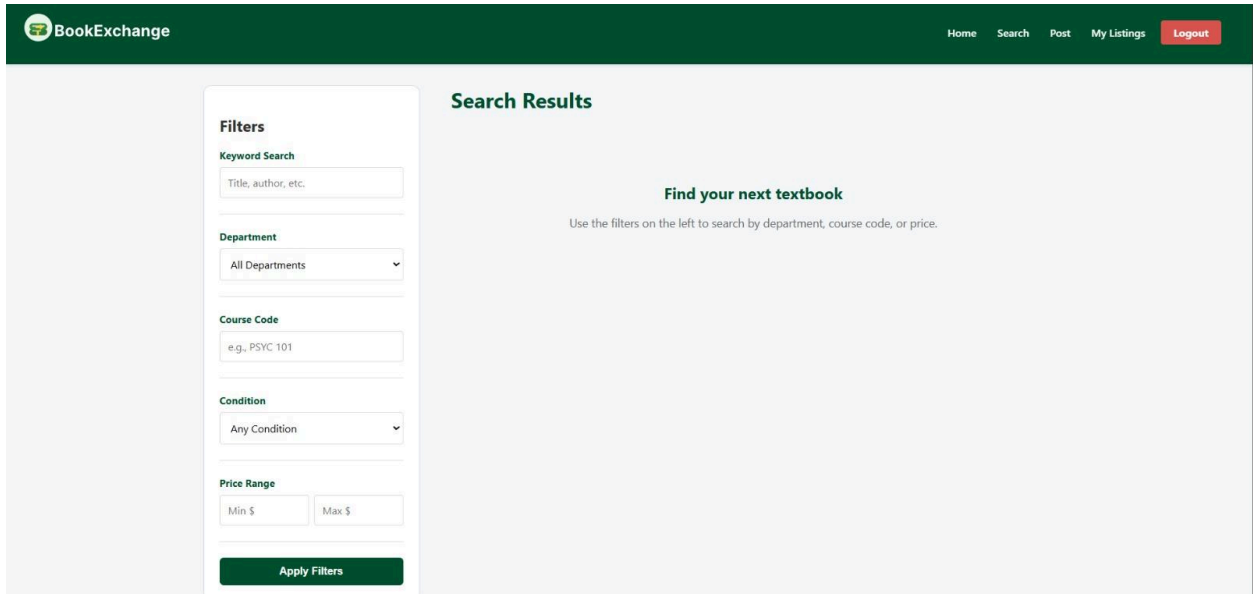
- Book Picture:** A placeholder image with a 'Choose File' button and a note 'Review will appear here'.
- Book Title:** A text input field with a placeholder '(e.g. 'Introduction to Software Engineering')'.
- Department:** A text input field with the placeholder 'ENSE'.
- Course Number:** A text input field with the placeholder '101'.
- Condition:** A dropdown menu with 'New' selected.
- Price:** A text input field with the placeholder '\$100'.
- Your Name/Handle *:** A text input field with a placeholder '(e.g. 'Chop P')'.
- Contact Detail:** A text input field with a placeholder 'Email or phone'.
- Optional Descriptions/Notes:** A large text area with a placeholder for additional information.

At the bottom of the form, there are two buttons: 'Publish Listing' (green) and 'Cancel' (gray).

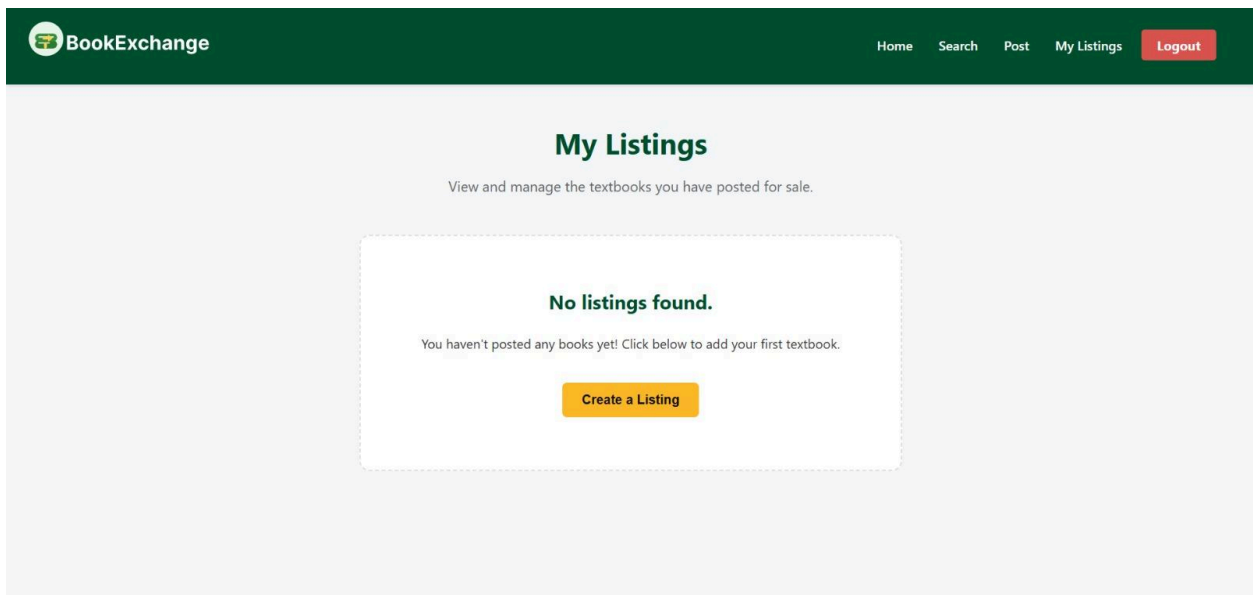
- Browse all available listings



- Search listings by course code or book title



- View full listing details, including seller contact information
- Mark a listing as sold
- Delete a listing



The team stayed true to the original scope throughout development. No features were cut, and no major pivots were required. The decision to use LocalStorage rather than a backend server proved sound: it eliminated hosting and database concerns entirely and allowed the team to focus on the user-facing experience.

Feedback Received and Its Impact

Feedback was gathered informally from classmates and through course check-ins with Dr. Maciag. The key themes that emerged were:

- Clarity of search: Early testers found it unclear which field they were searching. In response, the team added explicit placeholder labels to the search inputs and split them into separate course code and title filters.
- Sold listings visibility: There was confusion about whether sold items should still appear. The team decided to keep sold listings visible with a clear "SOLD" badge, so buyers could still contact sellers if needed.
- Mobile friendliness: Feedback pointed out that the layout did not scale well on phone screens. CSS adjustments were made to improve responsiveness, since most students would browse on mobile.

Team Exploration and Process

The team began with a brief ideation phase, identifying the textbook market gap as a real problem experienced by all four members. A simple wireframe was sketched to define the four key pages: a home/landing page, a listings browse page, a post-a-listing form, and a listing details page.

Work was divided by page and functionality. A shared JavaScript file (script.js) with a shared CSS file (style.css) served as the integration point. GitHub was used for version control throughout, with commits organized by feature. Regular syncs were held to catch integration issues early.

The key architectural decision (using LocalStorage as the data layer) had downstream effects on every feature. All listing data was serialized to JSON and stored under a single key, with each read/write cycle operating on the full array. This kept the data model simple and allowed rapid iteration.

Key User Interactions (Delivered MVP)

The delivered MVP supports the following primary user flows:

Seller Flow

- Navigate to "Post a Listing" from the home page.
- Fill out the form: book title, course code, condition (new/good/fair/poor), asking price, and email.
- Submit the form; the listing appears immediately in the Browse page.
- From the listing's detail page, mark it as sold or delete it.

Buyer Flow

- Navigate to "Browse Listings" from the home page.
- Use the search bar to filter by course code (e.g., "ENSE 271") or book title.

- Click on a listing card to view full details.
- Contact the seller via the displayed email address to arrange an in-person exchange on campus.

Conclusion

BookExchange successfully demonstrates that a real, usable software product can be delivered within a short academic semester using only client-side technologies. All goals were met, the MVP is functional, and the platform addresses a genuine need on campus.

The project reinforced key software engineering principles covered in ENSE 281. Keeping features simple and well-scoped allowed the team to deliver reliably. Feedback loops, even informal ones, made the product measurably better. And sharing a codebase via GitHub forced the team to communicate clearly, name things consistently, and document decisions.

If taken further, BookExchange would benefit from payment processing, a rating/review system, and a messaging system to keep interactions within the platform. These were intentionally deferred to keep the MVP achievable.

Self-Reflection

How Did We Feel About This Project?

Overall, the team felt proud of what was produced. Starting from a blank HTML file and arriving at a complete, working application within one semester was satisfying. The project felt meaningful because it solved a real problem, one we actually face as students.

What we liked most was the tangibility of the outcome. Unlike some academic exercises, BookExchange produced something that could genuinely be used. Watching a listing get posted and appear on the browse page felt like real software.

What we disliked was the limitations of LocalStorage as a data layer. Data does not persist across browsers or devices, which limits real-world usefulness. There were moments where we wished we could have used a simple backend, but we recognized the constraint was appropriate given the course scope.

What Is the Team Most Proud Of?

The team is most proud of delivering a complete, coherent product. Every feature listed in the scope was shipped. Nothing was abandoned or half-finished. The application loads, data persists within a session, the search works, and the listing lifecycle (post → view → sold/delete) functions end to end.

We are also proud of the consistency of the UI. Given that different team members worked on different pages, the visual coherence of the final product required deliberate coordination on shared CSS and naming conventions, and we got it right.

What Did We Learn About Ourselves?

Individually, each team member learned that the practices taught in ENSE 281 (clean naming, clear documentation, consistent code style) are not just theoretical ideals. When four people share a codebase, those practices directly affect how quickly you can understand and build on each other's work. Poor naming caused real friction; clear naming removed it.

As a team, we learned that early alignment on architecture pays off significantly. The decision on how LocalStorage would be structured (what keys, what schema) made all downstream work smoother. Teams that defer those decisions discover them the hard way through merge conflicts and broken features.

We also learned that scope management is a skill. The urge to add features (user accounts, photo uploads, ratings) was real and constant. Choosing not to was a deliberate decision that kept the project on track.

Obstacles, Frustrations, and How We Overcame Them

Honestly, the biggest challenge was not technical; it was time. Every team member was balancing this project alongside other courses, part-time jobs, and personal responsibilities outside of school. There were moments where progress was slower than expected, not because anyone was disengaged, but because life does not pause for a semester project.

As a team, we handled this by being patient and understanding with each other. If someone could not contribute as much in a given week, the rest of the team picked up the slack without making it a big deal. We stayed conscious of each other's time and never made anyone feel guilty for having other commitments. That mutual respect kept the atmosphere positive even when deadlines were close.

Being time-conscious also pushed us to work more efficiently. Instead of spending hours debating decisions, we made reasonable choices quickly and moved on. Scope creep was tempting, but we kept reminding ourselves that a finished, simple product beats an unfinished, complex one.

How Will We Apply These Learnings Going Forward?

The principles practised in this project (defining clear goals, building iteratively, seeking feedback, and keeping scope tight) apply directly to any future software work. Each team member now has a concrete, shareable project to reference when articulating these skills.

In future projects, we would invest earlier in a lightweight design document and make architectural decisions explicitly rather than implicitly. We would also build in more structured feedback checkpoints rather than relying on informal input.

BookExchange is a foundation. The version delivered here is a working MVP. If the team chose to continue, the next step would be to introduce a backend, enable real-time listings visible to all University of Regina students, and formalize it into something the campus community could actually use.